

PROGRAMMATION FONCTIONNELLE 2 : RÉSUMÉ À MI-PARCOURS février 2026 — Pierre Rousselin

On commence par faire le point sur les six semaines passées.

1 Semaine 1 : Variants et match

On a présenté les variants OCaml et `match`, avant de faire la même chose en Rocq.

```
(* Définition du type number à 3 constructeurs *)
type number = Float of float | Int of int | Error
let n1 = Float 2.3
let n2 = Int 0

(* Définition du type point à 1 seul constructeur *)
type point = Point of float * float
```

En OCaml, les constructeurs commencent par une majuscule (et réciproquement, tout identifiant commençant par une majuscule doit désigner un constructeur).

Les booléens (de type `bool`) sont notés `true` et `false`.

On peut faire un filtrage de motif (*pattern matching*) sur les variants et les booléens (et en fait aussi sur les autres types primitifs d'OCaml). Le filtrage de motif fait deux choses en même temps :

- raisonner par cas sur les différentes manières de construire un variant à plusieurs constructeurs et
- lier les arguments d'un constructeur à des variables dans la branche correspondant à ce constructeur.

```
let is_float n =
  match n with
  | Float _ -> true
  | _ -> false
let symm p =
  match p with
  | Point (x, y) -> Point (-. x, -. y)
```

Dans le cas d'un type à un seul constructeur, le *pattern matching* peut être fait directement au niveau du paramètre d'une fonction, ou dans une expression `let`.

```
let fst (Point (x, y)) = x
let snd p = let Point (x, y) = p in y
```

Un tel motif est dit « irréfutable » (car vrai dans tous les cas, vu qu'il y a un seul constructeur).

En Rocq, on a plutôt :

```
From Coq Require Import Reals BinInt.
Inductive bool := true | false.
Inductive unit := tt. (* ici c'est tt à la place de () *)
(* pour number, on va, par simplicité, éviter les types int et float et
   utiliser les nombres réels et les entiers relatifs *)
Inductive number :=
  | Real (r : R)
  | Int (i : Z)
  | Error.
Inductive point := Point (x y : R).
Definition symmetric (p : point) : point :=
  match p with
  | Point x y => Point (- x) (- y)
```

```

end.
Definition is_real (n : number) : bool :=
  match n with
  | Real r => true
  | Int i  => false
  | Error => false
  end.
(* Motif irréfutable en Rocq : prend une ' en plus *)
Definition other_sym '(Point x y) := Point y x.

```

Les commandes qui permettent de manipuler l'environnement global (ajouter des définitions de types, de fonctions, afficher un type ou le résultat du calcul, etc) sont appelées *vernaculaires*, commencent par une majuscule et se terminent par un point. Le langage des termes (**fun**, **match**, etc) est appelé Gallina.

En Rocq comme en OCaml, la méthode principale pour définir des fonctions est de faire un filtrage de motif sur les constructeurs.

Dans le TD1, on a principalement manipulé les booléens, avec des définitions et des réductions et des preuves par cas. Noter que les réductions peuvent aussi avoir lieu en présence de variables, dans ce cas, avec **simpl**, ce sera autant que possible (voir exercice 2 du TD1).

Dans le TP1, on a écrit des preuves, principalement par cas. Les tactiques et tacticielles rencontrées sont :

- **reflexivity** pour prouver une égalité où les deux membres sont identiques
- **simpl** pour réduire autant que possible dans le but
- **destruct b** pour faire une preuve par cas sur les différentes valeurs possibles de **b**.
- variantes : **destruct b1, b2** pour détruire **b1**, puis **b2**
- **rewrite règle** pour réécrire en utilisant une règle (une preuve d'égalité) dans le but
- la composition des tactiques avec ; comme dans **destruct b; simpl** pour appliquer **simpl** à tous les sous-buts engendrés par **destruct b**.

2 Semaine 2 : Prop

La généricité en OCaml se fait avec des variables spéciales de type **'a**, **'b**, etc qui apparaissent naturellement lorsqu'il n'y a pas de contrainte de type. Par exemple,

```

let id x = x (* val id : 'a -> 'a = <fun> *)

```

En Rocq, les types sont des termes comme les autres. Ils ont donc un type. Le type d'un type est appelé une sorte. La sorte des propositions (énoncés dits « logiques ») est appelée **Prop**. Une preuve d'une proposition **A** de type **Prop** est un habitant de **A**, c'est-à-dire un terme de type **A**.

Pour commencer on peut considérer **True** dans **Prop** (à ne pas confondre avec **true** dans **bool**) qui a un constructeur sans paramètre, noté **I** :

```

Inductive True : Prop := I.

```

La tactique **exact** termine une preuve en donnant un terme dont le type est celui du but. Pour prouver **True**, il suffit d'utiliser le constructeur **I**, ce qui est toujours possible vu qu'il n'a pas de paramètre.

```

Lemma True_is_True : True.
Proof. exact I. Qed.

```

Étant donné deux propositions **A** et **B** (c'est-à-dire deux types de sorte **Prop**), on peut créer la la conjonction de **A** et **B** :

```

Inductive and (A : Prop) (B : Prop) : Prop :=
  conj (hA : A) (hB : B)
Notation "A /\ B" := (and A B).

```

En français, si on a une preuve `hA` de `A`, une preuve `hB` de `B`, alors `conj hA hB` est une preuve de `A /\ B`.
 Pour la disjonction :

```
Inductive or (A : Prop) (B : Prop) : Prop :=
  or_introl (hA : A) | or_intror (hB : B).
Notation "A \/ B" := (or A B).
```

- Si j'ai une preuve `hA` de `A` alors `or_introl hA` est une preuve de `A \/ B`.
- Si j'ai une preuve `hB` de `B` alors `or_intror hB` est une preuve de `A \/ B`.

Tout ce qui a été dit précédemment sur `match` est encore valable avec des propositions, par exemple on a :

```
Lemma proj1 (A B : Prop) (h : A /\ B) : A.
Proof. exact (match h with conj hA _ => hA end). Qed.
Lemma and_comm (A B : Prop) (h : A /\ B) : B /\ A.
Proof. exact (match h with conj hA hB => conj hB hA end). Qed.
Lemma or_comm (A B : Prop) (h : A \/ B) : B \/ A.
Proof.
  exact (
    match h with
    | or_introl hA => or_intror hB
    | or_intror hB => or_introl hA
    end).
Qed.
```

Fournir directement le terme dans un cas simple comme ceux ci-dessus est intéressant pour comprendre la théorie des types sous-jacente. Il existe aussi des tactiques spécialisées qui rapprochent l'écriture des preuves de la déduction naturelle. Ici on a

- `left` ou `right` pour prouver (introduire) un « ou »
- `split` pour prouver (introduire) un « et »
- `destruct h as [hA hB]` pour utiliser (éliminer) un « et »
- `destruct h as [hA | hB]` pour utiliser (éliminer) un « ou »

Par exemple :

```
Lemma and_comm (A B : Prop) (h : A /\ B) : B /\ A.
Proof.
  destruct h as [hA hB].
  split.
  - exact hB.
  - exact hA.
Qed.
Lemma or_comm (A B : Prop) (h : A \/ B) : B \/ A.
Proof.
  destruct h as [hA | hB]
  - right.
    exact hA.
  - left.
    exact hB.
Qed.
```

La tactique `destruct` correspond à `match` et les tactiques `left`, `right` et `split` à l'utilisation d'un constructeur. Ces preuves sont finalement les mêmes que celles avec terme de preuve ci-dessus.

Enfin, ces tactiques correspondent à des règles d'introduction et d'élimination en déduction naturelle :

$$\begin{array}{c}
 \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge\text{E-g}) \\
 \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge\text{E-d}) \\
 \frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee\text{I-g}) \\
 \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee\text{I-d}) \\
 \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge\text{I}) \\
 \frac{}{\Gamma, A \vdash A} \text{Ax} \\
 \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P} (\vee\text{E})
 \end{array}$$

Pour ce qui est de l'implication, c'est primitif en Rocq. En fait, le type $A \rightarrow B$ n'est rien d'autre que le type des fonctions de A vers B (peu importe que A et B soient dans **Prop** ou autre).

Prouver une implication, revient donc à fournir une fonction. Utiliser une implication, à appliquer cette fonction :

```

Lemma impl_refl (A : Prop) : A -> A.
Proof. exact (fun h => h). Qed.
Lemma modus_ponens (A B : Prop) (h : A -> B) (hA : A) : B.
Proof. exact (h hA). Qed.

```

Côté tactiques on a :

- **intros** h . qui remplace **exact** (**fun** $h \Rightarrow \dots$).
- **apply** h . pour appliquer une fonction (avant de lui fournir un argument éventuel).

Ici, ça donne :

```

Lemma imp_refl (A : Prop) : A -> A.
Proof.
  intros hA. (* on a hA : A dans les hypothèses et le but est A *)
  exact hA.
Qed.
Lemma modus_ponens (A B : Prop) (h : A -> B) (hA : A) : B.
Proof.
  apply h. (* le but devient A *)
  exact hA.
Qed.

```

À leur tour, **intros** et **apply** correspondent respectivement à la règle d'introduction et la règle d'élimination de l'implication en déduction naturelle.

$$\begin{array}{c}
 \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (\rightarrow\text{I}) \\
 \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\rightarrow\text{E})
 \end{array}$$

En TD, nous avons mis en parallèle ces trois aspects :

- terme de preuve pour fabriquer un terme d'un certain type (programmation fonctionnelle)
- preuve avec tactiques (preuves formelles)
- arbre de dérivation en déduction naturelle

En TP, on a continué ce travail tactiques/termes de preuve avec en plus, pour des preuves plus courtes :

- retour sur la composition des tactiques (avec **;**)
- réutilisation de noms dans différents cas d'un **destruct**
- utilisation de l'automatisation légère **assumption** pour factoriser les différents **exact**.

3 Semaine 3 : tuples et curryfication, False

En OCaml, on a, par exemple,

```
let proj1 x y = y
(* val proj1 : 'a -> 'b -> 'a = <fun> *)
```

En fait, mettre les variables à gauche de = est du sucre syntaxique (une facilité d'écriture pour les programmeurs) pour

```
let proj1 = fun x -> fun y -> x
```

Donc proj1 est une fonction qui retourne une fonction. D'ailleurs le type 'a -> 'b -> 'a de cette fonction doit être considéré avec des parenthèses à droite : 'a -> ('b -> 'a).

Ceci est en contraste avec les tuples, types produits qui agrègent différents types en un seul, comme dans

```
let fst = fun c -> match c with (x, y) -> x
(* val fst : 'a * 'b -> 'a = <fun> *)
(* ou avec filtrage de motif (irréfutable) directement dans l'argument : *)
let snd = fun (x, y) -> y
```

C'est pareil en Rocq, par exemple avec

```
Definition add_tuple := fun c => match c with (x, y) => x + y end.
Check add_tuple. (* add_tuple : nat * nat -> nat *)
(* Avec motif irréfutable dans l'argument : *)
Definition add_triple := fun '(x, y, z) => x + y + z.
Check add_triple. (* add_triple : nat * nat * nat -> nat *)
```

Mais en Rocq, le type produit n'est pas primitif, c'est prod dans Type et and dans Prop.

En général on préfère les fonctions curryfiées pour formaliser les fonctions de plusieurs variables (mais en gardant les types produits en conclusion des fonctions/implications).

On retient que :

- Le type $A \rightarrow B \rightarrow C$ est en fait $A \rightarrow (B \rightarrow C)$
- qui correspond à si j'ai du A et du B, je peux avoir du C.

Exemple avec tactiques et terme de preuve :

```
Lemma baz1 (A B C : Prop) : A -> B -> (A -> B -> C) -> C.
Proof.
  intros hA. (* on suppose hA : A et hB : B *)
  intros hABC. (* on suppose hABC : A -> B -> C *)
  apply hABC. (* génère deux buts ! *)
  - exact hA.
  - exact hB.
Qed.
Lemma baz2 (A B C : Prop) : A -> B -> (A -> B -> C) -> C.
Proof.
  exact (fun hA hB hABC => hABC hA hB).
Qed.
```

En TD, on a travaillé sur la curryfication et l'instanciation partielle de fonctions curriées. On a vu les fonction curry et uncurry ainsi que les formules logiques dont ces fonctions sont les preuves.

Le TP3 fait travailler avec la proposition False, qui n'a pas de constructeur et de laquelle on peut déduire n'importe quoi.

4 Semaine 4 : forall et types dépendants

On commence par remarquer que les types de `id`, `proj1` et `proj2` en OCaml qui sont

```
let id = fun x -> x
(* val id : 'a -> 'a = <fun> *)
let proj1 = fun x _ -> x
(* val proj1 : 'a -> 'b -> 'a = <fun> *)
let proj2 = fun _ y -> y
(* val proj2 : 'a -> 'b -> 'b = <fun> *)
```

peuvent être vus comme universellement quantifiés sur les variables de type :

$$\begin{aligned} \text{id} &: \forall \alpha, \quad \alpha \rightarrow \alpha \\ \text{proj1} &: \forall \alpha, \forall \beta, \quad \alpha \rightarrow \beta \rightarrow \alpha \\ \text{proj2} &: \forall \alpha, \forall \beta, \quad \alpha \rightarrow \beta \rightarrow \beta \end{aligned}$$

En Rocq, les types étant des termes comme les autres, ces quantifications sont explicites :

```
Definition id (A : Type) : A -> A := fun x => x.
Definition proj1 (A B : Type) : A -> B -> A := fun x _ => x.
Definition proj2 (A B : Type) : A -> B -> B := fun _ y => y.
Check id. (* forall A : Type, A -> A *)
Check proj1. (* forall A B : Type, A -> B -> A *)
Check proj2. (* forall A B : Type, A -> B -> B *)
```

On parle de *types dépendants* (ici, de variables de types) : le type de `id`, `proj1` et `proj2` dépend de son ou ses premiers paramètres.

On voit ensuite les types produits et sommes (ou unions) :

```
Inductive prod (A B : Type) := pair (a : A) (b : B).
Check prod. (* prod : Type -> Type -> Type *)
Check pair. (* pair : forall (A B : Type), A -> B -> prod A B *)
Inductive sum (A B : Type) := inl (a : A) | inr (b : B).
Check sum. (* sum : Type -> Type -> Type *)
Check inl. (* inl : forall (A B : Type), A -> sum A B *)
Check inr. (* inr : forall (A B : Type), B -> sum A B *)
```

qui ne sont rien d'autre que les analogues dans `Type` de `and` (pour le produit) et `or` (pour la somme) dans `Prop`.

Prouver un `forall` revient donc encore une fois à fournir une fonction, dont le type est dépendant comme ci-dessous :

```
Lemma foo1 : forall (A : Prop), A -> A.
Proof.
  exact (fun (A : Prop) => fun (hA : A) => hA).
Qed.
(* Idem avec sucre syntaxique *)
Lemma foo2 : forall (A : Prop), A -> A.
Proof.
  exact (fun (A : Prop) (hA : A) => hA).
Qed.
```

et en fait la notation `A -> B` n'est rien d'autre qu'une notation pour `forall _ : A, B` où le type d'arrivée (`B`) ne dépend pas de la valeur de l'argument (ici de type `A`) de la fonction.

Pour prouver un `forall`, on utilise donc encore la tactique `intros` pour nommer des paramètres.

```

Lemma foo5 : forall (A : Prop), A -> A.
Proof.
  intros A. (* Soit A une proposition. *)
  intros hA. (* On suppose hA : A *)
  exact hA.
Qed.
Lemma foo6 : forall (A : Prop), A -> A.
Proof.
  intros A hA.
  exact hA.
Qed.

```

Cette introduction du « pour tout » correspond encore à celle de la déduction naturelle :

$$\frac{\Gamma, x : T \vdash P}{\Gamma \vdash \forall x : T, P} (\forall I)$$

Comme précédemment, utiliser un « pour tout » n'est rien d'autre qu'appliquer une fonction.

```

Lemma impl_refl : forall (A : Prop), A -> A.
Proof. intros A hA. exact hA. Qed.
Lemma bar1 : forall (A B : Prop), (A -> B -> A) -> (A -> B -> A).
Proof.
  intros A B.
  exact (impl_refl (A -> B -> A)).
Qed.
Check or_introl. (* or_introl : forall (A B : Prop), A -> A \ / B *)
Lemma bar2 : forall A : Prop, A -> A \ / A.
Proof.
  intros A hA.
  (* point technique : or_introl a ses premiers arguments
   * qui sont implicites *)
  (* le @ avant permet de récupérer la version sans implicites *)
  exact (@or_introl A A hA).
Qed.

```

On peut le faire plus progressivement avec la tactique `specialize` :

```

Lemma bar3 : forall (A B : Prop), (A -> B -> A) -> (A -> B -> A).
Proof.
  intros A B.
  specialize impl_refl as H.
  (* H : forall A : Prop, A -> A *)
  specialize (H (A -> B -> A)).
  (* H : (A -> B -> A) -> A -> B -> A *)
  exact H.
Qed.
Lemma bar4 : forall A : Prop, A -> A \ / A.
Proof.
  intros A hA.
  specialize or_introl as oinl.
  (* oinl : forall A B : Prop, A -> A \ / B *)
  specialize (oinl A A) as H.
  (* H : A -> A \ / A *)
  specialize (H hA).
  (* H : A \ / A *)
  exact H.
Qed.

```

Au passage, on peut utiliser `specialize` avec n'importe quelle fonction.

Finalement, on a besoin d'encore plus de dépendance pour pouvoir quantifier nos propositions sur, disons, des entiers, des booléens. Ceci peut se faire en Rocq avec un filtrage de motif à type dépendant, par exemple dans :

```
Definition false_or_42 (b : bool) :=
  match b return (if b then bool else nat) with
  | true => false
  | false => 42
end.
Check false_or_42. (* forall b : bool, if b then bool else nat *)
```

Ici c'est la clause `return` qui donne le type commun des deux branches du `match`.

On a donc un système de types dit « dépendants » dont l'expressivité est au moins celle de la déduction naturelle.

Le TD4 revient sur :

- les types dépendants de types en OCaml, avec les fonctions `ignore`, `const`, `flip`, etc
- d'autres types dépendants en Rocq, dont le type `ex` pour le quantificateur « il existe »

Le TP4 revient sur les tactiques `intros` et `specialize` (ou, à la place de `specialize`, instancier explicitement ou non une preuve d'un type dépendant) et sur le quantificateur « il existe ».

5 Semaine 5 : types rékursifs

On redécouvre les types rékursifs avec les entiers de Peano et les listes (génériques) :

```
Inductive nat : Set := 0 : nat | S : nat -> nat.
Inductive list (A : Type) : Type :=
  nil : list A | cons : A -> list A -> list A.
```

Notations :

- usuelles en décimal pour les entiers naturels de Peano :

```
Check 0. (* 0 : nat *)
Check 42. (* 42 : nat *)
```

- Comme dans OCaml, avec les bons réglages/imports :

```
From Coq Require Import List.
Import ListNotations.
Check []. (* liste vide *)
Check 42 :: []. (* [42] : list nat *)
Check [4; 3; 2; 1]. (* [4; 3; 2; 1] : list nat *)
```

On définit des fonctions par filtrage de motifs sur les constructeurs.

```
Definition is_empty {A : Type} (l : list A) :=
  match l with
  | [] => true
  | _ => false
end.
Fixpoint app {A : Type} (l0 l1 : list A) :=
  match l0 with
  | [] => l1
  | h :: q => h :: (app q l1)
end.
Fixpoint add (n m : nat) :=
  match n with
  | 0 => m
  | S p => S (add p m)
end.
```


Ici, la commande vernaculaire **Fixpoint** remplace **Definition** pour les définitions récursives (comme **let rec** en OCaml). Point crucial pour la définition récursive : se demander comment reconstruire la solution pour $h :: q$ (resp. pour $S\ p$) si on la connaît pour q (resp. pour p).

Ensuite, une preuve par induction est une preuve par cas où l'on a en plus, lorsque le constructeur a un argument du type récursif lui-même (ici **cons** pour une liste, **S** pour un naturel), une hypothèse d'induction sur cet argument.

La tactique utilisée est **induction**. Exemples :

Lemma `app_nil_r` {A : Type} : **forall** (l : list A), `app l []` = l.

Proof.

```

intros l.
induction l as [| h q IH].
- (* cas l = [] *)
  simpl.
  reflexivity.
- (* cas l = h :: q
   avec l'hypothèse d'induction :
   IH : app q [] = q *)
  simpl.
  rewrite IH. (* changer (app q []) en q dans le but *)
  reflexivity. (* h :: q = h :: q *)

```

Qed.

Lemma `add_0_r` : **forall** (n : nat), `n + 0` = n.

Proof.

```

intros n. induction n as [| p IH].
- simpl. (* 0 + 0 se réduit en 0 *)
  reflexivity. (* 0 = 0 ok *)
- (* n = S p où p satisfait
   IH : p + 0 = p *)
  simpl. (* S p + 0 se réduit en S (p + 0) *)
  rewrite IH. (* On utilise IH pour changer
               p + 0 en p dans le but *)
  reflexivity. (* S p = S p, ok *)

```

Qed.

Fixpoint `length` {A : Type} (l : list A) :=

```

match l with
| [] => 0
| _ :: q => S (length q)
end.

```

Lemma `length_app` {A : Type} (l l' : list A) :

`length (app l l')` = `length l` + `length l'`.

Proof.

```

induction l as [| h q IH].
- simpl. reflexivity.
- simpl. rewrite IH. reflexivity.

```

Qed.

C'est la même syntaxe que **destruct**, sauf qu'il faut nommer l'hypothèse d'induction.

Pour finir, on voit que la preuve par induction elle-même est encore une fonction récursive. Par exemple sur les entiers :

Fixpoint `nat_ind` (P : nat -> Prop)

```

(h0 : P 0) (hered : forall k, P k -> P (S k)) (n : nat) :=
match n with
| 0 => h0
| S p => hered p (nat_ind P h0 hered p)
end.

```

Ces preuves sont générées lorsqu'on définit les types avec **Inductive** et utilisées par la tactique **induction**.

Le TD5 est important. On y voit comment écrire des preuves par induction, notamment, lisiblement, sur papier, en justifiant chaque étape.

Le TP5 fait travailler l'induction et les définitions récursives.

6 Semaine 6

Au programme du partiel, seulement la première partie qui récapitule :

- les règles de base obtenues par une étape de réduction, comme **add_0_1**, **add_succ_1**, **app_nil_1**, etc
- comment les utiliser dans une preuve en Rocq avec **rewrite** pour s'affranchir de **simpl**
- comment les utiliser dans une « preuve papier très détaillée ».

Le reste du CM6 et les TD et TP 6 ne sont pas au programme de ce premier partiel.

7 Liste des tactiques

Liste des tactiques par semaine de cours :

1. Semaine 1 : **exact**, **destruct**, **simpl**, **reflexivity**, **rewrite**
2. Semaine 2 : **destruct** (un « ou », un « et »), utilisation des *intro patterns*, **split**, **left** et **right**, **apply**, **assumption**
3. Semaine 3 : **destruct** une preuve de **False**, **exfalse**, **unfold** (pour la négation)
4. Semaine 4 : **specialize**, **exists** (prouver un « il existe »), **destruct** (pour utiliser un « il existe »)
5. Semaine 5 : **induction** et plus de **rewrite** (notamment avec **-** devant la règle pour « de droite à gauche »).